redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

Team Name :Ankit Singh

Team Leader Name :Ankit Singh

Problem Statement : Build an AI system that ranks candidates the way a great recruiter would — not by matching keywords, but by actually understanding who fits the role.

Solution Overview

- I read the JD as a rubric, not a keyword list, and rank candidates by what they've actually built. My pipeline has four stages: ensemble semantic retrieval narrows 100k→985, a local LLM judge reads each shortlisted candidate's career history like a recruiter would, a listwise rerank orders the top of the list, and a deterministic gate removes fabricated profiles.
- What sets my approach apart from traditional candidate matching: this dataset is adversarial by design. The skills list is engineered noise — every one of 133 skills appears ~12,000 times regardless of role, so a Marketing Manager can carry "Fine-tuning LLMs." Keyword and skill matching is a deliberate trap. So I discard the skills array entirely and read the free-text career descriptions instead. That's what lets me surface real engineers hiding under plain titles and reject buzzword-stuffed non-fits — the exact judgment the JD asks for.

JD Understanding & Candidate Evaluation

- The key requirements I extracted from the JD: production experience with embeddings-based retrieval, vector-DB / hybrid search, and — most importantly — having shipped at least one end-to-end ranking, search, or recommendation system at a product (not services) company. Plus 5–9 years of experience, strong Python, and ranking-evaluation literacy (NDCG/MRR/MAP). The JD is also emphatic about disqualifiers: services-only careers, title-chasers, framework-only "LangChain-calling-OpenAI" profiles, research-only, CV/speech-only, and the unavailable.
- The signals I treat as most important, in order: the free-text career descriptions (what someone actually built) first, then title trajectory, then availability. I deliberately ignore two signals — the skills array (engineered noise) and the templated summary (self-contradicting). My whole read of "fit beyond keywords" is the gap between what a profile says and what it means: someone who built a recommendation system at a product company is a fit even without the buzzwords; a Marketing Manager with a perfect AI skill list is not.

Ranking Methodology

- Retrieve: I use ensemble dense retrieval — BGE-large-en-v1.5 and E5-large-v2, fused with Reciprocal Rank Fusion. I score each candidate as similarity to a positive query minus 0.5× similarity to a negative query, where the negative query encodes the JD's explicit disqualifiers. I run no title filter, so genuine builders under non-ML titles survive into the shortlist.
- Score: a local Qwen2.5-7B model acts as the recruiter-judge. It reads each candidate's evidence text plus a block of pre-verified structured facts and returns a fit tier (0–4), a fit score, and fact-grounded reasoning.
- Rank: I sort primarily by the judge's tier, then within a tier by `0.70·judge_score + 0.30·structured_fit`, all multiplied by a bounded availability factor (0.85–1.0). Finally I run a listwise LLM rerank across the top 40 (three passes, mean-rank aggregated) to get the top-10 ordering right, since NDCG@10 is 50% of the score. The combination principle is tier-first: a strong-on-paper candidate never leapfrogs a genuinely higher-tier one.

Explainability & Data Validation

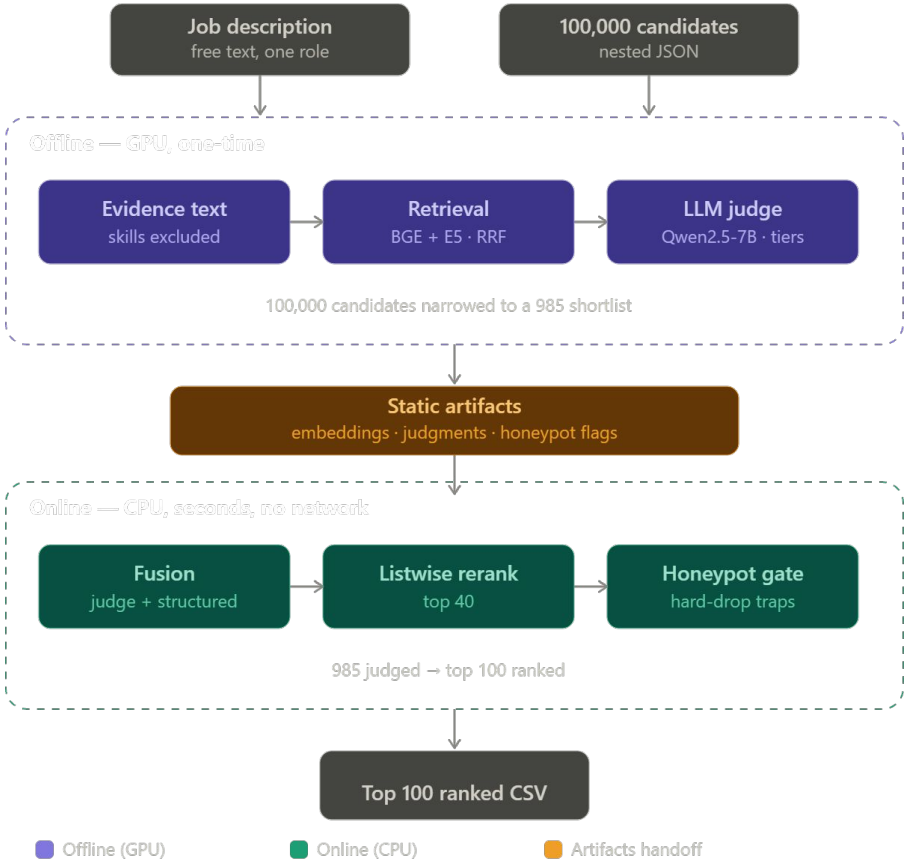
- Every ranked candidate carries a one-line recruiter note that I build from the judge's extracted evidence — a real company, a real system they built, a concrete metric — plus an honest stated concern where one exists (44 of my 100 rows carry an explicit concern). I prevent hallucination by grounding every reasoning string only in facts pulled from that candidate's own record; nothing is free-generated, and I dedupe so all 100 read distinctly.
- For suspicious profiles: I built a deterministic honeypot gate that catches "subtly impossible" fabrications through timeline-consistency math (skill duration vs tenure, tenure vs experience, date sanity). I went this route because I proved empirically that keyword-perfect fakes fool both the embeddings and the LLM judge — the impossibility lives in the dates, not the prose — so the defense has to be deterministic, not model-based. During my audit I found and closed a second honeypot family (inflated experience) that my first detector was structurally blind to, taking detection from ~55% to ~86%.

End-to-End Workflow

- JD (free text) → I distill it into positive and negative rubric queries → I embed all 100k candidates on their evidence text (skills excluded) → ensemble retrieval plus a build-signal rescue produces a 985-candidate shortlist → deterministic honeypot flagging → the LLM judge scores all 985 → I fuse tiers with structured fit and availability → listwise rerank of the top 40 → honeypot hard-drop → top 100 with reasoning → validated output file.
- All the heavy compute happens offline; the ranking step itself is CPU-only and runs in about half a second.

System Architecture

- The core of my design is the offline/online split. Offline (GPU, one-time): I build the embeddings, flag honeypots, and run the LLM judge — all baked into small static artifacts. Online (CPU, seconds, no network): I load those artifacts, fuse the scores, apply the honeypot gate, and emit the ranking. This is why I can use a heavy 7B judge and two large embedding models while still meeting a strict CPU-only, sub-5-minute ranking budget — the expensive reasoning is precomputed, and the ranking step just reads it.



Results & Performance

- I validated ranking quality against an independent hand-labeled gold set — labeled by a different model than my judge, so the pipeline isn't grading itself. My ablations show the full pipeline beats every degraded version: full vs judge-only is +0.166 composite, full vs embeddings-only is +0.179, which proves each component earns its place. A key measured finding: honeypots score highest on raw embeddings, which is the direct evidence that my deterministic gate is necessary rather than optional.
- My final top-100 is 44 tier-4 and 56 tier-3 candidates, with zero honeypots, and the output is byte-reproducible across runs. On compute: only the ranking step is budget-bound (≤5 min, CPU, no network), and it loads precomputed artifacts and finishes in about half a second — the spec explicitly permits offline precomputation.

Technologies Used

I chose BGE-large and E5-large as complementary embedders because ensembling them recovers fits that either one alone misses, and fused them with RRF. For the judge I used Qwen2.5-7B run fully locally via Ollama — zero API cost, fully reproducible, and it fits on my 8GB RTX 4060 Ti. The CPU ranking step uses only pandas, numpy, and pyarrow; I used pydantic to enforce structured judge output, and plain deterministic Python for the honeypot gate. I selected everything for local reproducibility, no external dependencies at rank time, and quality concentrated where the score is actually won — the top 10.

Submission Assets

- GitHub repository: https://github.com/ankitsinghh007/redrob-candidate-ranking
- Approach deck (this PDF)
- Ranked output file: `ankitsingh058622_1300_ranking.xlsx` (top 100 candidates)

redrob | H2S HACK2SKILL

INDIA.RUNS

Build what next India runs on

THANK YOU